

Meeting Leech Detector (MLD) - Complete Project Summary

1. Project Overview

Meeting Leech Detector (MLD) is an AI-powered telemetry and analytics platform designed to measure true meeting engagement. Its goal is to identify active contributors, track focus metrics, and eliminate wasted time during remote or hybrid meetings. It provides a comprehensive solution for organizations to monitor if employees are actively engaged in meetings (e.g., using Zoom, Microsoft Teams, WebEx) by tracking their active desktop windows and webcam usage.

2. System Architecture

The platform is built using a dual-tier architecture:

- **Backend Server:** A custom-built Java HTTP Server that handles REST API requests, database interactions, and telemetry collection.
- **Desktop Agent:** A lightweight Java application (MLD-Agent.jar) running on employee devices to quietly capture telemetry data and stream it to the backend.
- **Frontend Web Application:** A responsive HTML/JS/CSS portal styled with Bootstrap 5, allowing managers and employees to interact with the system.

3. Detailed Component & File Breakdown

3.1 Backend (Java src/)

The backend is a pure Java implementation that handles HTTP requests directly without relying on heavy frameworks like Spring Boot.

- **src/api/ApiServer.java**
 - Acts as the main web server using `com.sun.net.httpserver.HttpServer`.
 - Handles all routing and REST API endpoints.
 - **Endpoints include:**
 - `/login`, `/google-login`: Authentication.
 - `/google-signup-org`, `/google-signup-emp`: Registration.
 - `/api/start`, `/api/stop`: Manager endpoints to start or stop a meeting session.
 - `/api/join`, `/api/leave`: Employee endpoints to join or leave a session.
 - `/api/track`: Receives live telemetry ticks from the desktop agent.
 - `/api/analytics`, `/api/engagement`: Provide aggregated data for manager dashboards.
 - `/api/export`: Exports data as CSV.
- **src/database/DatabaseHelper.java**
 - Handles data persistence.
 - **PostgreSQL Support:** Connects to a PostgreSQL database if `DATABASE_URL` or environment variables are provided.
 - **In-Memory Fallback:** If PostgreSQL is unavailable, it gracefully falls back to high-performance `ConcurrentHashMap` in-memory structures for storing Users, Organizations, Devices, Sessions, and Telemetry.

- **Tables/Entities:** organizations, users, devices, sessions, telemetry (records window title, webcam state, engagement score), and notifications.

3.2 Desktop Agent (Java src/agent/ & src/monitor/)

The agent is downloaded by employees and runs in the background.

- **src/agent/MldAgent.java**
 - The entry point for the desktop client.
 - Prompts the user to log in or uses a saved configuration token (.mld_agent.properties).
 - Runs a continuous background loop (using ScheduledExecutorService) every 5 seconds to poll the server for an active session.
 - When an active session is detected, it collects telemetry (window title, webcam status) and sends a tick payload to /api/track.
- **src/monitor/ActiveWindowTracker.java**
 - Uses JNA (Java Native Access) to interface with the Windows OS.
 - **getActiveWindowTitle():** Identifies the title of the foreground window to see if the user is focused on a meeting app (Zoom, Teams, WebEx) or doing something else.
 - **isWebcamActive():** Checks the Windows Registry (HKCU\Software\Microsoft\Windows\CurrentVersion\CapabilityAccessManager\ConsentStore\webcam) to verify if the camera is currently in use.

3.3 Frontend Web Application

The frontend is a static web app communicating with the Java backend via REST API.

- **HTML Pages (Root & pages/)**
 - **index.html:** The login portal for both managers and employees. Supports email/password and Google OAuth Sign-In.
 - **organization-signup.html:** Registration page for new companies/managers.
 - **employee-signup.html:** Registration page for employees to join an existing organization using an Org Code.
 - **pages/manager-dashboard.html:** The admin panel. Managers can generate a "Session Code" to start a meeting, view live engagement metrics, and receive alerts.
 - **pages/employee-dashboard.html:** The employee view. They can enter a session code to join a meeting. It also provides instructions and downloads for the Desktop Agent.
 - **pages/analytics.html, pages/reports.html, pages/alerts.html:** Secondary manager views for deep-dive historical data, CSV exports, and rule-based alerts (e.g., someone minimizing the meeting).
- **JavaScript (js/)**
 - **js/api.js:** A wrapper utility for fetch API. Automatically handles authentication headers (bearer tokens) and JSON parsing.
 - **js/main.js:** The core UI logic. Contains handlers for Google Auth, form submissions, DOM manipulation (showing/hiding UI elements), and rendering real-time graphs using Chart.js on the dashboard.
 - **js/export.js:** Likely handles the logic for downloading CSV reports from the dashboard.
- **CSS (css/)**
 - Custom styling overriding and extending Bootstrap 5. Employs modern UI trends like glassmorphism (.glass-card), glowing orbs for backgrounds, and responsive layouts.

3.4 Build & Execution Scripts

The root directory contains several utility scripts for managing the lifecycle of the application:

- **run.bat / build-agent.bat:** Windows batch scripts to compile the Java code and package the MLD-Agent.jar.
- **start-ml-d-agent.bat / .ps1:** Scripts provided to employees to easily launch the Java agent in the background.
- **stop-ml-d-agent.bat / .ps1:** Scripts to gracefully terminate the agent.
- **run-silent-agent.vbs:** A VBScript to run the agent completely hidden without a command prompt window.
- **check_db.bat:** A utility script to verify database connectivity.
- **Dockerfile:** Contains the configuration to containerize and deploy the Java backend to cloud services (like Render, AWS, or Heroku).

4. Workflows

4.1 Onboarding

1. A Manager creates an organization via `organization-signup.html` and receives an **Org Code**.
2. Employees register via `employee-signup.html` using the **Org Code** to link their accounts.
3. Employees log in and download the MLD-Agent.jar via the Employee Dashboard.

4.2 Meeting Session Execution

1. The Manager logs into the dashboard and clicks **"Start Meeting Session"**. The system generates a unique **Session Code** (e.g., MLD123).
2. Employees enter MLD123 into their Employee Dashboard to mark themselves as present.
3. The background **Desktop Agent** on employee machines detects the active session and automatically begins polling.
4. Every 5 seconds, the agent checks if the user's foreground window is the meeting app (e.g., Zoom) and if the webcam is on.
5. Telemetry is sent to the backend. The Manager's Dashboard automatically updates with live charts showing the organization's overall engagement percentage, active participants, and flags employees who are tabbed out.
6. The Manager stops the session, and the Desktop Agents automatically return to standby mode, ceasing telemetry collection.

5. Summary

The MLD project is a full-stack, enterprise-grade telemetry solution. It leverages low-level OS APIs via Java to silently collect behavioral metrics and presents them through a polished, real-time web dashboard. It is designed to be highly scalable with PostgreSQL, yet portable with its in-memory fallback, making it robust for tracking meeting efficiency.

6. Key Development Challenges & Solutions

Throughout the development of MLD, several technical hurdles were encountered and resolved:

- **The "Java Not Installed" Problem (Auto-JRE Download):** The MLD-Agent.jar requires Java to run. To prevent setup failures for employees without Java, the `start-ml-d-agent.ps1` script was rewritten to include

an **auto-JRE downloader**. It automatically fetches a portable Java Runtime from Eclipse Adoptium, extracts it, and runs the agent seamlessly.

- **Messy Background Execution & Launching:** Running the Java agent via the command line left annoying console windows open. A suite of launcher scripts (.bat, .ps1, .vbs) was created to start the agent entirely hidden in the background, along with scripts to gracefully kill the process when the meeting ends.
- **File Distribution & Missing Scripts:** Employees downloading the agent often missed the helpful startup scripts. The build process (build-agent.bat) was updated to bundle everything into an MLD-Agent.zip distribution package, which is now automatically served by the backend.
- **Data Mixing Across Organizations (Multi-Tenancy):** Initially, the analytics backend was pulling telemetry data across the entire database. The ReportGenerator was patched to enforce strict multi-tenant data isolation using SQL queries bound to orgId.

7. Open Discussion / Questions for Professor

Topic for feedback: Alternatives to a Downloadable Desktop Agent

Currently, the solution relies on employees downloading and running a background desktop agent to track active windows and webcam states. This successfully captures side-activities and engagement for remote workers globally, but requiring a manual download can create friction during onboarding.

Question for Professor: *While maintaining the ability to capture OS-level window states and side-activities, what could be alternative proposed solutions or architectural approaches instead of forcing employees to download and run a local agent? Are there other modern methods (e.g., progressive web apps, browser extensions, or enterprise MDM policies) that could achieve this same level of telemetry without a manual download?*